

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Definition von „Problemen“

Definition

Ein **Problem** ist eine (abzählbar) unendliche Menge möglicher Instanzen (Inputs) zusammen mit einer Frage.

Arten:

- **Entscheidungsproblem** erwartet als Antwort „ja“ oder „nein“.
- **Funktionsproblem**: Welches Ergebnis liefert eine Funktion f bei Eingabe x ? Die Antwort ist $f(x)$.
- **Optimierungsproblem** sucht den besten Wert (Minimum oder Maximum) einer Funktion $f(x)$ zur gegebenen Instanz x , z. B. die Länge des kürzesten Pfads zwischen zwei Knoten.
- **Suchproblem** verlangt eine konkrete Lösung zur Instanz, z. B. einen Pfad von u nach v .
- **Aufzählproblem** fordert alle Lösungen zur Instanz, z. B. alle zyklensfreien Pfade von u nach v .
- **Zählproblem** ermittelt die Anzahl der Lösungen, z. B. wie viele zyklensfreie Pfade es von u nach v gibt.

Warum unendlich viele Instanzen?

Ein Lösungsverfahren soll für beliebige Instanzen funktionieren. Gäbe es nur endlich viele, könnte man jede Lösung vorab berechnen und abspeichern („Look-up-Tabelle“). Unendlich viele Instanzen erlauben den Einsatz mathematischer Werkzeuge zur Analyse von Problem-Schwierigkeit.

Definition eines Algorithmus

Definition

Ein **Algorithmus** für ein Problem P beschreibt eine Abfolge von Rechenschritten, mit denen sich jede Instanz von P lösen lässt.

Ein Algorithmus muss:

- immer terminieren – also nach endlich vielen Schritten enden,
- die richtige Antwort liefern,
- aus einfachen Schritten bestehen, die eine Maschine ausführen kann,
- verständlich beschrieben sein, damit Menschen ihn nachvollziehen können.

Programmiersprachen dienen dazu, Rechenschritte formal zu beschreiben – also Programme zu schreiben.

Programme

In imperativen Sprachen (z. B. C, Java) werden die Rechenschritte **explizit** beschrieben.

In deklarativen Sprachen (z. B. Haskell, Prolog, SQL) werden die Rechenschritte **implizit** beschrieben. Der Interpreter oder Compiler erzeugt daraus die konkreten Schritte.

Programme sollen:

- für Menschen verständlich sein,
- und von Computern ausführbar.

Algorithmen vs. Programme

Wenn ein Programm für alle zulässigen Instanzen terminiert und immer eine korrekte Antwort liefert, dann ist es ein Algorithmus. Wenn diese Eigenschaften nicht zutreffen durch z.B. Endlosschleifen oder Abstürze, handelt es sich nicht um einen gültigen Algorithmus.

Berechenbarkeit

Um über die Lösbarkeit von Problemen sprechen zu können, wird ein präzises Modell für Berechenbarkeit benötigt. In der Informatik wird dafür meist eine Programmiersprache als formales Rechenmodell verwendet.

Gibt es für ein bestimmtes Problem einen Algorithmus?

- Für viele Probleme: Ja, ein Algorithmus existiert
- Für einige natürliche Probleme: Nein, es existiert kein Algorithmus und P ist unentscheidbar

Unsere Programmiersprache: SIMPLE

SIMPLE ist ein vereinfachtes Modell prozeduraler Programmiersprachen. Es enthält:

- **Variablen** (z. B. vom Typ: String, Integer, Boolean, Void)
- **Zuweisungen** (z. B. $x := y + z$)
- **if/then/else**-Anweisungen
- **while**-Schleifen
- **for**- und **repeat**-Schleifen (können durch while ersetzt werden)
- **return**-Anweisungen
- Prozeduren und Funktionen

Eingaben für ein SIMPLE-Programm:

- Eine Liste von Werten: $L = (V_1, V_2, \dots, V_n)$
- Oder ein einzelner String (z. B. als ASCII oder Bitfolge)

Ausgabe für ein SIMPLE-Programm:

Ein Wert beliebigen Typs, der mit einer return-Anweisung zurückgegeben wird.

Eignung von SIMPLE als Berechenbarkeitsmodell

SIMPLE ist ein geeignetes Modell für Berechenbarkeit. Alle Algorithmen, die sich in Programmiersprachen wie Java, C oder Python ausdrücken lassen, können auch in SIMPLE formuliert werden. Die Sprache ist mächtig genug, um sogar Interpreter oder Compiler anderer Sprachen – etwa eine Java Virtual Machine – umzusetzen. Daraus folgt: Wenn sich ein Problem nicht mit einem SIMPLE-Programm lösen lässt, dann existiert auch grundsätzlich kein Algorithmus dafür unabhängig von der gewählten Programmiersprache.

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Berechenbarkeit von Funktionen

Definition

Eine Funktion heißt **berechenbar**, wenn es einen Algorithmus gibt, der für jeden Input x den Funktionswert $f(x)$ korrekt berechnet.

- Die Eingaben können beliebige Objekte sein: Zahlen, Strings, Graphen, Datenbanken ...
- Diese müssen sich als **endliche Strings** über einem **endlichen Alphabet** (z. B. ASCII oder $\{0,1\}$) codieren lassen.

Typische Funktionen:

$$f: \mathbb{N} \rightarrow \mathbb{N} \qquad f: \Sigma^* \rightarrow \Sigma^* \qquad f: \mathbb{N} \rightarrow \{0,1\} \qquad f: \Sigma^* \rightarrow \{0,1\}$$

Funktionsproblem $\rightarrow$ **Entscheidungsproblemen** : $\{0,1\}$, {ja, nein}, {wahr, falsch}

Entscheidbarkeit

Definition

Ein Entscheidungsproblem P ist **entscheidbar**, wenn ein Algorithmus A für jede beliebige Instanz x des Problems P als Input nimmt und die korrekte Antwort (ja/nein, 1/0, wahr/falsch) liefert.

Eine Funktion ist **berechenbar**, beziehungsweise ein Entscheidungsproblem ist **entscheidbar**, wenn ein SIMPLE-Programm existiert, das für jede zulässige Eingabe korrekt terminiert. Viele Funktionen und Entscheidungsprobleme erfüllen diese Bedingung. Es gibt jedoch auch Funktionen, die nicht berechenbar sind, sowie Probleme, für die kein Algorithmus existiert. Solche Probleme nennt man **nicht entscheidbar**.

Existenz von unentscheidbaren Problemen

SIMPLE-Programme sind endliche Zeichenketten über einem endlichen Alphabet, zum Beispiel dem ASCII-Zeichensatz. Auch die Eingaben von Entscheidungsproblemen lassen sich als solche Zeichenketten darstellen. Entscheidungsprobleme entsprechen daher Funktionen der Form $\Sigma^* \rightarrow \{0,1\}$

Theorem

Die Menge aller Strings Σ^* ist **abzählbar unendlich** $\Rightarrow$ Es existiert eine bijektive Abbildung: $g: \Sigma^* \rightarrow \mathbb{N}$
Die Menge aller Funktionen $\mathbb{N} \rightarrow \{0,1\}$ ist **überabzählbar** $\Rightarrow \Sigma^* \rightarrow \{0,1\}$ ist überabzählbar.

Daraus folgt: Es gibt überabzählbar viele Entscheidungsprobleme, aber nur abzählbar viele SIMPLE-Programme. Es muss also Entscheidungsprobleme geben, für die kein SIMPLE-Programm existiert und somit auch kein Algorithmus. Solche Probleme bezeichnet man als **unentscheidbar**.

Abzählbarkeit von Σ^*

Beweis

Angenommen, das Alphabet ist endlich: $\Sigma = \{a_1, a_2, \dots, a_k\}$. Σ^* ist die Menge **aller endlichen Strings** über dem Alphabet Σ . Diese Menge ist **abzählbar unendlich**.

Aufzählstrategie:

- 1) Zuerst nach Stringlänge sortieren: leere Strings, Strings der Länge 1, 2, 3, ...
- 2) Innerhalb gleicher Länge: lexikographische (systematische) Reihenfolge

Durch diese geordnete Aufzählung lässt sich eine Funktion $g: \Sigma^* \rightarrow \mathbb{N}$ (abzählbar unendlich) definieren, die jedem String seine Position in der Aufzählung zuordnet. Diese Funktion ist **bijektiv**, da jedem String genau eine natürliche Zahl zugewiesen wird (Injektivität) und jede natürliche Zahl auf genau einen String verweist (Surjektivität). Das zeigt: Σ^* ist **abzählbar**!

Aufzählung:

Strings	Anzahl	Positionen
ϵ (leerer String)	1	0
$a_1, \dots, a_k$	k	$1, \dots, k$
$a_1 a_1, a_1 a_2, \dots, a_k a_k$	k^2	$k+1, \dots, k^2+k$
$a_1 a_1 a_1, a_1 a_1 a_2, \dots, a_k a_k a_k$	k^3	$k^2+k+1, \dots, k^3+k^2+k$
etc.		

Überabzählbarkeit von $\{0,1\}^{\mathbb{N}}$

Beweis (Cantor'sches Diagonalverfahren)

Angenommen, die Menge aller Funktionen $f: \mathbb{N} \rightarrow \{0,1\}$ sei **abzählbar**. Dann ließen sich alle diese Funktionen wie bei einer Aufzählung von Strings durchnummerieren: $f_0, f_1, f_2, f_3, \dots$

Jede Funktion f_i lässt sich als unendliche Folge von Nullen und Einsen darstellen:

$$f_0 = (f_0(0), f_0(1), f_0(2), \dots) \quad f_1 = (f_1(0), f_1(1), f_1(2), \dots) \quad \dots$$

Nun konstruiert man eine neue Funktion f' indem wir die Diagonale dieser Aufzählung betrachten und an jeder Stelle den Wert umdrehen:

$$f'(n) = \begin{cases} 1, & \text{falls } f_n(n) = 0 \\ 0, & \text{falls } f_n(n) = 1 \end{cases} \quad \text{bzw.} \quad f'(n) = 1 - f_n(n)$$

Die so konstruierte Funktion f' unterscheidet sich in mindestens einer Stelle von **jeder** Funktion f_n in der Liste an Position n . Daher kann f' **nicht** in der ursprünglichen Aufzählung enthalten sein. Dies widerspricht der Annahme, dass die Menge aller Funktionen $\mathbb{N} \rightarrow \{0,1\}$ abzählbar ist. Somit ist die Menge aller Funktionen $\mathbb{N} \rightarrow \{0,1\}$ **überabzählbar**.

Bemerkung:

Cantor zeigt mit dem Argument auch, dass die Menge der reellen Zahlen $\mathbb{R}$ überabzählbar ist.

Aufzählung von $\{0,1\}^{\mathbb{N}}$:

f_0	=	a 00	a_{01}	a_{02}	a_{03}	a_{04}	...
f_1	=	a_{10}	a 11	a_{12}	a_{13}	a_{14}	...
f_2	=	a_{20}	a_{21}	a 22	a_{23}	a_{24}	...
$\vdots$	=	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

Probleme über Programme

Einige Probleme betreffen das Verhalten von Programmen selbst:

- Läuft ein Programm Π mit einem gegebenen Input I in eine Endlosschleife?
- Terminiert Π überhaupt für alle möglichen Inputs?

Solche Algorithmen wären sehr hilfreich zur Überprüfung der Korrektheit von Programmen und zur Lösung mathematischen Problemen.

Goldbachsche Vermutung

Die Goldbachsche Vermutung besagt, dass **jede gerade Zahl größer als 2 die Summe zweier Primzahlen ist**. Man kann ein Programm schreiben, das dies prüft, indem es für jede gerade Zahl n testet, ob sie sich als Summe zweier Primzahlen darstellen lässt. Ein solches Programm überprüft die Aussage schrittweise für $n = 4, 6, 8, 10, \dots$ und erhöht n jeweils um 2, solange die Eigenschaft erfüllt ist. Wenn die Vermutung stimmt, läuft das Programm endlos weiter. Wenn sie falsch ist, terminiert es bei der ersten Gegenbeispiel-Zahl.

Daraus ergibt sich: Die Goldbachsche Vermutung ist genau dann wahr, wenn das Programm nicht terminiert.

Computergestützter Beweis

Angenommen, es existiert ein Programm Π_n das entscheidet, ob ein beliebiges Programm terminiert. Dann könnte man mit Hilfe von Π_n die Gültigkeit der Goldbachschen Vermutung algorithmisch überprüfen. Man lässt Π_n auf das Programm `testConjecture()` laufen.

- Falls Π_n mit „nein“ antwortet, terminiert `testConjecture()` nicht und die Vermutung ist wahr.
- Falls Π_n mit „ja“ antwortet, existiert eine gerade Zahl $n > 2$ die nicht die Summe von 2 Primzahlen ist.

Obwohl man nicht weiß, wie lange Π_h für die Entscheidung benötigt, liefert es im Erfolgsfall eine korrekte Antwort. Damit könnte man nicht nur die Goldbachsche Vermutung, sondern auch viele weitere mathematische Probleme lösen – vorausgesetzt, man kann die Termination algorithmisch entscheiden

Unentscheidbarkeit des Halteproblems

Problem HALTEPROBLEM

Instanz: Programm Π und Input String I .

Frage: Terminiert das Programm Π auf dem Input I ?

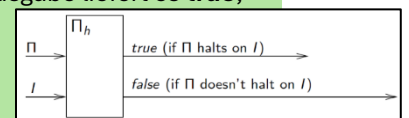
Wir nehmen an, dass ein Algorithmus Π_h existiert, der das Halteproblem entscheidet. Der Beweis erfolgt **indirekt**, d.h. aus der Annahme, dass Π_h für das Halteproblem existiert (entscheidbar), wird ein **Widerspruch** konstruiert. Dabei wird angenommen, dass sowohl das Programm Π_h als auch der Input I als Strings gegeben sind. Das Halteproblem ist unentscheidbar.

Beweis durch Widerspruch

Annahme: Es gibt ein Programm Π_h , das entscheidet, ob ein Programm Π mit einem Input I terminiert.

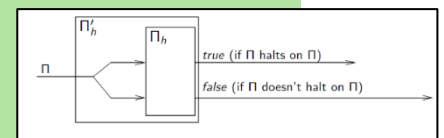
Π_h nimmt zwei Strings als Eingabe: den Quellcode von Π und den Input I . Als Ausgabe liefert es **true**, falls Π auf I terminiert und **false**, falls nicht.

Boolean $\Pi_h(\text{String } \Pi, \text{String } I)$ /* code to check if Π terminates on I . */



Mit Π_h wird ein neues Programm Π'_h definiert, das nur **einen** String (nämlich Π) als Eingabe erhält, diesen dupliziert und dann Π_h ausführt. Damit prüft Π'_h , ob ein Programm Π terminiert, wenn es sich selbst als Input erhält.

Boolean $\Pi'_h(\text{String } \Pi)$
return $\Pi_h(\Pi, \Pi)$;



Dann wird ein weiteres Programm Π''_h konstruiert, das das Verhalten von Π'_h umdreht: Falls $\Pi'_h(\Pi)$ **true** ergibt, läuft Π''_h in eine Endlosschleife. Falls **false**, terminiert es.

Nun wird Π'_h mit seinem eigenen Quellcode als Input aufgerufen. Dabei ergeben sich zwei Fälle:

- Wenn $\Pi''_h(\Pi'_h)$ terminiert, dann folgt aus der Definition von Π''_h , dass es nicht terminiert → Widerspruch.
- Wenn $\Pi''_h(\Pi'_h)$ nicht terminiert, dann folgt aus der Definition, dass es terminiert → ebenfalls Widerspruch.

Da beide Fälle zum Widerspruch führen, kann das angenommene Programm Π_h nicht existieren.

Weitere Beispiele unentscheidbarer Probleme

Problem KORREKTHEIT

- Instanz: Gegeben ist ein Programm Π sowie zwei Strings I_1 und I_2 .
- Frage: Gefragt ist, ob Π bei Eingabe I_1 den Output I_2 liefert

Diese Frage ist unentscheidbar, da sie impliziert, dass entschieden werden muss, ob Π auf I_1 terminiert.

Problem ERREICHBARER CODE

- Instanz: Gegeben ist ein Programm Π und eine Programmzeile n .
- Frage: Gefragt ist, ob es eine Eingabe I gibt, bei der Π die Zeile n ausführt.

Auch dieses Problem ist **unentscheidbar**, da sich das Halteproblem darauf reduzieren lässt, wenn man n als letzte Zeile von Π wählt.

Bemerkung:

Unentscheidbarkeit kann oft durch intuitive Argumente gezeigt werden. Um dies formaler zu machen, wird später der Begriff der **Reduktion** eingeführt.

Teil 1.3: Semi-Entscheidbarkeit

Definition

Ein Entscheidungsproblem P heißt **semi-entscheidbar**, wenn es ein Programm Π gibt, das wie folgt arbeitet:

- Das Programm Π nimmt eine Instanz I von P als Eingabe.
- Wenn I eine „ja“-Instanz ist, liefert Π das Ergebnis **true**.
- Wenn I eine „nein“-Instanz ist, liefert Π das Ergebnis **false** oder **terminiert**.

Das bedeutet:

- Das Programm funktioniert korrekt auf allen positiven Instanzen.
- Auf negativen Instanzen darf es endlos laufen.

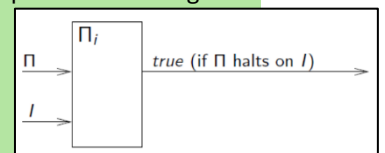
Semi-Entscheidbarkeit des Halteproblems

Beweis

Es lässt sich ein Interpreter-Programm Π_i definieren, das eine beliebige Instanz des Halteproblems als Eingabe erhält, also den Quellcode eines Programms Π sowie einen Input I .

Das Programm Π_i simuliert die Ausführung von Π auf I .

- Falls die Simulation terminiert, gibt Π_i **true** zurück.
- Falls die Simulation nicht terminiert, läuft auch Π_i **endlos**.



Somit erkennt Π_i alle positiven Instanzen des Halteproblems korrekt und zeigt, dass das Problem semi-entscheidbar ist.

Semi-Entscheidbarkeit des Korrektheitsproblems

Problem ERREICHBARER CODE

- Instanz: Gegeben ist ein Programm Π sowie zwei Strings I_1 und I_2 .
- Frage: Gefragt ist, ob Π bei Eingabe I_1 den Output I_2 liefert

Das **Korrektheitsproblem** ist **semi-entscheidbar**.

Beweis

Man konstruiert ein Interpreter- Programm Π_i , das eine beliebige Instanz des Problems als Eingabe erhält – also den Quellcode von Π_i , sowie I_1 und I_2 .

Das Programm Π_i simuliert die Ausführung von Π auf I_1 .

Falls Π bei dieser Eingabe terminiert und das Ergebnis gleich I_2 ist, gibt Π_i **true** aus, sonst **false**.

Wenn die Simulation nicht terminiert, läuft auch Π_i **endlos**.

Semi-Entscheidbarkeit des Erreichbarer-Code-Problems

Problem ERREICHBARER CODE

- Instanz: Gegeben ist ein Programm Π und eine Programmzeile n .
- Frage: Gefragt ist, ob es eine Eingabe I gibt, bei der Π die Zeile n ausführt.

Für ein gegebenes Paar (I, i) , also ein möglicher Input I und eine Schrittzahl i , kann man einen Interpreter verwenden, der Π für i Schritte simuliert. Dieser entscheidet, ob innerhalb dieser i Schritte die Zeile n erreicht wurde.

Beweis

Man zählt alle Paare (I, i) auf und prüft, ob Π bei Eingabe I die Zeile n in i Schritten erreicht.

- Falls (Π, n) eine positive Instanz ist, findet man garantiert ein passendes Paar und gibt true zurück.
- Falls (Π, n) eine negative Instanz ist, läuft die Aufzählung endlos.

Einschub: Aufzählung und Abzählbarkeit

Im Zusammenhang mit dem Semi-Entscheidungsverfahren für das Korrektheitsproblem stellt sich die Frage, wie man alle Paare (I, i) aufzählen kann. Eine einfache verschachtelte Schleife über alle Inputs $I \in \Sigma^*$ und alle $i \in \mathbb{N}$ funktioniert nicht, da das Programm bei einem nicht-terminierenden Input I hängen bleiben kann. Dadurch würden weitere Paare gar nicht erst getestet – also eben nicht alle (I, i) .

Das führt zur allgemeinen Frage: Wie kann man das kartesische Produkt $M_1 \times M_2$ aufzählen, wenn M_1 und M_2 jeweils abzählbar unendlich sind? Oder anders gefragt: Ist $M_1 \times M_2$ ebenfalls abzählbar, wenn beide Ausgangsmengen es sind?

Abzählbarkeit (Fortsetzung)

- **Definition:** Eine Menge M heißt **abzählbar**, wenn sie endlich ist oder es eine *bijektive Abbildung* $f: M \rightarrow \mathbb{N}$ gibt.

Beobachtungen:

- Ist M unendlich, muss eine bijektive Abbildung $f: M \rightarrow \mathbb{N}$ konstruiert werden.
- Einfacher: Stattdessen kann man $g: \mathbb{N} \rightarrow M$ als Aufzählung definieren.
- Es reicht auch, eine *injektive* Abbildung $f: M \rightarrow \mathbb{N}$ zu finden.
- **Intuition:** Gibt es eine injektive Abbildung, dann gilt $|M| \leq |\mathbb{N}|$, also höchstens abzählbar.
- **Formale Begründung:** g ist injektiv und surjektiv, also bijektiv. Aus einer injektiven Abbildung f konstruiert man eine bijektive Abbildung g mit $g(a) = |\{i \mid i < f(a)\}|$

Abzählbare unendliche Mengen

- $M = \mathbb{N} \cup \{a\}$: $f(a) = 0, f(n) = n + 1$ für alle $n \in \mathbb{N} \rightarrow$ injektiv
- $M = \mathbb{N} \cup \{a_1, \dots, a_k\}$: $f(a_i) = i - 1, f(n) = n + k$ für alle $n \in \mathbb{N}$
- $M = \{0, 1\} \times \mathbb{N}$: $f: M \rightarrow \mathbb{N}$ mit $f(i, n) = 2n + i$ für alle $i \in \{0, 1\}$ und $n \in \mathbb{N}$
- $M = \mathbb{N} \times \mathbb{N}$ oder $M = M_1 \times M_2$: Cantor'sches Abzählprinzip

(0,0),
(1,0), (1,1), (0,1),
(2,0), (2,1), (2,2), (1,2), (0,2)
(3,0), (3,1), (3,2), (3,3), (2,3), (1,3), (0,3),
etc.

→ zeigt: auch kartesische Produkte solcher Mengen sind wieder abzählbar unendlich.

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\begin{pmatrix} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \begin{pmatrix} 1 & 4 & 9 & 16 & \dots \\ 2 & 3 & 8 & 15 & \dots \\ 5 & 6 & 7 & 14 & \dots \\ 10 & 11 & 12 & 13 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

Daraus folgt:

Theorem

Das ERREICHBARER-CODE Problem ist semi-entscheidbar.

Bemerkung: Cantor hat so die Abzählbarkeit von $\mathbb{Q}$ gezeigt.

Weitere semi-entscheidbare Probleme

- **Instanz:** Eine Formel ϕ der Prädikatenlogik erster Stufe.
- **Frage:** Ist ϕ gültig?

→ Das Entscheidungsproblem ist **semi-entscheidbar**.

Wiederholung: Terme & Formeln

- **Terme:**
 - Konstantensymbole (z. B. a, b, c)
 - Variablen (z. B. x, y, z)
 - Zusammengesetzte Terme: z. B. $f(t_1, \dots, t_\alpha)$ für ein Funktionssymbol f mit Arität α und Terme $t_1, \dots, t_\alpha$
- **Formeln:**
 - Atome: $P(t_1, \dots, t_\alpha)$ für ein Prädikatensymbol P mit Arität α und Terme $t_1, \dots, t_\alpha$
 - Zusammengesetzt: $\neg\phi, \phi \wedge \psi, \phi \vee \psi, (\exists x) \phi, (\forall x) \phi$ für Formeln ϕ, ψ und Variable x

Beweis Unentscheidbarkeit

- Reduktion vom Halteproblem auf das Entscheidungsproblem.
- Wird später genauer über Turingmaschinen formalisiert.

Beweis Semi-Entscheidungsverfahren

Vollständige Kalküle existieren für die Prädikatenlogik → Jede gültige Formel hat eine endliche Ableitung.

Beispiel: Hilbert-Kalkül

Axiome: $A \rightarrow (B \rightarrow A), (\neg A \rightarrow \neg B) \rightarrow (B \rightarrow A), [A \rightarrow (B \rightarrow C)] \rightarrow [(A \rightarrow B) \rightarrow (A \rightarrow C)], \dots$

Modus Ponens: Aus A und $A \rightarrow B$ folgt B

(wenn man bereits Formeln A und $(A \rightarrow B)$ abgeleitet hat, dann kann man daraus B ableiten)

Vorgehen:

- Alle möglichen Ableitungen im Kalkül durchprobieren.
- Wenn ϕ gültig ist, wird es irgendwann abgeleitet → true.
- Wenn ϕ ungültig ist, läuft Verfahren endlos → nicht entscheidbar, aber semi-entscheidbar.

Teil 1.4: Komplement

Komplement eines Entscheidungsproblems

- Ein Entscheidungsproblem P fragt „ja oder nein“ für unendlich viele Instanzen.
- Das **Komplement** ($\text{co-}P$) fragt dasselbe, aber mit **verneinter Antwort**.
- Das $\text{co-}P$ Problem von $\text{co-}P$ ist wieder P (doppelte Verneinung).

Graph-Erreichbarkeit:

- Instanz: Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.
- Frage: Gibt es im Graph G **einen** Pfad von u nach v ?

Nicht-Erreichbarkeit (co-Erreichbarkeit):

- Instanz: Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.
- Frage: Gibt es im Graph G **keinen** Pfad von u nach v ?

Theorem

Wenn ein Problem P entscheidbar ist, dann ist auch $\text{co-}P$ entscheidbar.

Beweis

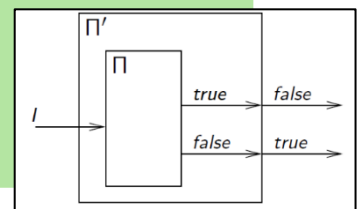
Wenn P entscheidbar ist, dann gibt es ein Programm Π , das für alle positiven Instanzen von P true und für alle negativen Instanzen von false zurückgibt.

Wir können Π zu einem neuen Programm Π' ändern, indem wir einfach den Ausgabewert invertieren.

Dann ist Π' eine Entscheidungsprozedur für $\text{co-}P$.

Boolean Π' (String I)

```
if  $\Pi(I)$  = true  
  then return false  
  else return true;
```



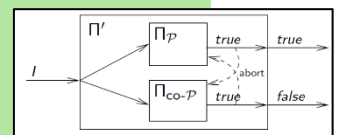
Theorem

Wenn sowohl P als auch $\text{co-}P$ **semi-entscheidbar** sind, dann ist P **entscheidbar**.

Beweis

- Da P semi-entscheidbar ist, gibt es ein Programm Π_P , das auf den positiven Instanzen von P terminiert und true ausgibt.

- Da $\text{co-}P$ semi-entscheidbar ist, gibt es ein Programm $\Pi_{\text{co-}P}$, das auf den positiven Instanzen von $\text{co-}P$ (d.h.: auf den negativen Instanzen von P) terminiert und true ausgibt.



Wir definieren ein Programm Π' , das Π_P und $\Pi_{\text{co-}P}$ parallel ausführt. Eines der beiden Programme terminiert garantiert. Dann liefert Π' den entsprechenden Output und stoppt das andere Programm.

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Frage: Gibt es Probleme, die nicht einmal semi-entscheidbar sind?

Theorem

Sei P ein Entscheidungsproblem, so dass P unentscheidbar und semi-entscheidbar ist.
Dann ist $\text{co-}P$ nicht semi-entscheidbar

Beweis

Angenommen, $\text{co-}P$ ist semi-entscheidbar. Das heißt, sowohl P als auch $\text{co-}P$ ist semi-entscheidbar. Dann folgt aus dem vorigen Theorem, dass P entscheidbar ist. Widerspruch zur Annahme (1).

Aus dem Theorem folgt, dass die $\text{co-}P$ -Probleme von Halteproblem, Korrektheit, Erreichbarer-Code und Entscheidungsproblem **nicht semi-entscheidbar** sind.

Gibt es Probleme, sodass weder P noch $\text{co-}P$ semi-entscheidbar ist?

Problem SELBER OUTPUT

Instanz: Programme Π_1, Π_2 und Input String I .

Frage: Haben Π_1 und Π_2 auf Input I das gleiche Verhalten?

Das heißt: entweder terminieren Π_1 und Π_2 bei Input I und liefern das gleiche Ergebnis oder beide Programme terminieren nicht bei Input I ?

Wir können positive Instanzen, bei denen keines der Programme Π_1 und Π_2 auf Input I terminiert, nicht erkennen. Denn wenn wir das könnten, wäre Nicht-Termination (sprich: das co-Halteproblem) semi-entscheidbar. Ein Widerspruch!

Wir können aber auch nicht negative Instanzen erkennen, bei denen eines der Programme Π_1 und Π_2 auf I terminiert und das andere nicht. Denn wenn wir das könnten, würde das wiederum bedeuten, dass Nicht-Termination (sprich: das co-Halteproblem) semi-entscheidbar wäre. Ein Widerspruch!

Problem PROGRAMM ÄQUIVALENZ

Instanz: 2 Programme Π_1, Π_2 (Input jeweils String über einem vorgegebenen Alphabet Σ)

Frage: Sind Π_1 und Π_2 äquivalent?

Das Problem SELBER OUTPUT vergleicht das Verhalten von 2 Programmen auf einem Input, wohingegen das Problem PROGRAMM ÄQUIVALENZ das Verhalten von 2 Programmen auf unendlich vielen Inputs vergleicht. Wenn also schon das einfachere Problem und sein co-Problem nicht semi-entscheidbar sind, dann auch das allgemeinere Problem.

Problem UNIVERSELLES HALTPROBLEM

Instanz: Programme Π (Input String über einem vorgegebenen Alphabet Σ)

Frage: Hält Π auf jedem beliebigen Input String $I \in \Sigma^*$?

Wir können positive Instanzen nicht erkennen, weil wir für unendlich viele Instanzen $I \in \Sigma^*$ überprüfen müssten, ob Π auf Input I hält.

Wir können negative Instanzen nicht erkennen, weil wir dafür die Nicht-Termination von Π auf irgendeinem String I erkennen müssten. Das entspricht aber dem co-Halteproblem .

Bemerkung: Wir haben für die Nicht-Semi-Entscheidbarkeit bis jetzt nur die Intuition beschrieben. Für einen formalen Beweis benötigen wir den Begriff der **Reduktion**.

Teil 1.6: Reduktionen

Allgemeine Idee

Angenommen, wir wollen ein neues Problem A lösen (z. B. durch Entwicklung eines Algorithmus). Und angenommen, wir wissen bereits, wie man ein verwandtes Problem B löst (z. B. es existiert ein Algorithmus für B).

Dann besteht die Idee darin, A auf B zu reduzieren, d. h.:

- Wir formen eine Instanz von A so um, dass wir sie mit dem Lösungsverfahren von B behandeln können.
- Das bedeutet, wir nutzen die Lösung von B, um A zu lösen.

Wenn diese Strategie funktioniert, schreiben wir:

$A \leq B \rightarrow$ **Problem A ist mindestens so leicht lösbar wie Problem B**

Idee für Berechenbarkeit und Komplexität

Angenommen, es gelingt uns nicht, ein neues Problem B zu lösen, d. h.: wir finden keinen Algorithmus oder kein Semi-Entscheidungsverfahren für B und angenommen, wir wissen, dass ein verwandtes Problem A schwer zu lösen ist, folgt für A gibt es bewiesenermaßen keinen effizienten Algorithmus bzw. nicht einmal ein Semi-Entscheidungsverfahren.

Dann ist die Idee:

Wir konstruieren einen (hypothetischen) Algorithmus B, der auf einem Algorithmus für A basiert.

$A \leq B \rightarrow$ **Problem B ist mindestens so schwer lösbar wie Problem A**

Motivation:

- Eine Reduktion von A nach B sollte einfacher sein als die Probleme A und B selbst.
- Sonst könnte die Reduktion die Komplexität von A „verstecken“, und ein sinnvoller Vergleich der Schwierigkeit von A und B wäre nicht möglich.

Typische Anforderungen:

- In der Berechenbarkeitstheorie: Reduktionen müssen berechenbar sein
- In der Komplexitätstheorie: Reduktionen müssen effizient berechenbar sein
z.B. in polynomieller Zeit $O(n^k)$ für Instanzgröße n und Konstante $k \geq 1$

Idee für Turing Reduktionen

Wir konstruieren ein Lösungsverfahren für Problem A, das ein **Lösungsverfahren für Problem B als Unterprozedur** verwendet. Das **Lösungsverfahren für Problem A** besteht also aus einem **Steuerungsprogramm**, das die Unterprozedur für Instanzen von Problem B “beliebig” oft aufrufen darf.

Ressourcenbeschränkung: Das Steuerungsprogramm selbst muss ein (effizienter) Algorithmus sein. Im Fall einer Beschränkung auf polynomielle Zeit, heißt so eine Reduktion “**Cook Reduktion**”.

Idee für Many-One Reduktionen

Definiere eine **Funktion** R von der Menge der Instanzen von Problem A auf die Menge der Instanzen von Problem B, d.h.: jede Instanz x von Problem A wird auf eine Instanz R(x) von Problem B abgebildet.

Wenn man nun die Instanz R(x) mit einem Lösungsverfahren für Problem B löst, dann ist das Ergebnis für die Instanz R(x) bereits das korrekte Ergebnis für die Instanz x des Problems A $\rightarrow$ **x und R(x) sind äquivalent.**

Für Entscheidungsprobleme A und B bedeutet das: x ist eine positive Instanz von A, R(x) ist eine positive Instanz von B.

Ressourcenbeschränkung: Die Funktion R muss (effizient) berechenbar sein. Im Fall einer Beschränkung auf polynomielle Zeit, heißt so eine Reduktion “Karp Reduktion”.

Bemerkung: Many-One Reduktionen sind ein Spezialfall von Turing Reduktionen: die Unterprozedur für Problem B wird exakt einmal aufgerufen und die Berechnung endet unmittelbar danach.

Klassische Entscheidungsprobleme der Aussagenlogik

Problem SAT

- Instanz: Aussagenlogische Formel ϕ
- Frage: Ist ϕ erfüllbar? d.h.: gibt es eine Wahrheitsbelegung I für die aussagenlogischen Variablen in ϕ , sodass ϕ von I erfüllt wird?

Problem 3-SAT

- Instanz: Aussagenlogische Formel ϕ in 3-CNF, d.h.: konjunktive Normalform, bei der jede Klausel aus höchstens 3 Literalen besteht.
- Frage: Ist ϕ erfüllbar?

Problem 2-SAT

- Instanz: Aussagenlogische Formel ϕ in 2-CNF, d.h.: konjunktive Normalform, bei der jede Klausel aus höchstens 2 Literalen besteht.
- Frage: Ist ϕ erfüllbar?

Reduktion von 2-SAT auf Graph-Erreichbarkeit

Theorem

Das 2-SAT Problem lässt sich mittels Turing Reduktion auf das Graph-Erreichbarkeit Problem reduzieren.

Das Graph-Erreichbarkeitsproblem ist effizient lösbar (sogar linear). Da $A = 2\text{-SAT}$ und $B = \text{Graph-Erreichbarkeit}$, gilt $A \leq B$ mit effizientem Verfahren für B . Die Reduktion ist polynomiell, somit läuft auch das Verfahren für 2-SAT in polynomialer Zeit.

Notation

Für ein Literal α , d.h. eine aussagenlogische Variable x_i oder ihre Negation $\neg x_i$, gilt:
Das duale Literal von α wird mit $\bar{\alpha}$ bezeichnet.

- Wenn $\alpha = x_i$, dann ist $\bar{\alpha} = \neg x_i$
- Wenn $\alpha = \neg x_i$, dann ist $\bar{\alpha} = x_i$

Von der Formel zum Graphen

Sei ϕ eine beliebige Instanz von 2-SAT mit Variablen $X = \{x_1, \dots, x_n\}$.

Daraus definieren wir den Graphen $G_\phi = (V, E)$:

- Knotenmenge: $V = \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n\}$
→ enthält alle Literal-Knoten, die aus den Variablen gebildet werden können.
- Kantenmenge: Für jede Klausel $(\alpha \vee \beta)$ in ϕ fügen wir die Kanten $(\bar{\alpha}, \beta)$ und $(\bar{\beta}, \alpha)$ hinzu.
→ G_ϕ repräsentiert Implikationen zwischen Literalen.

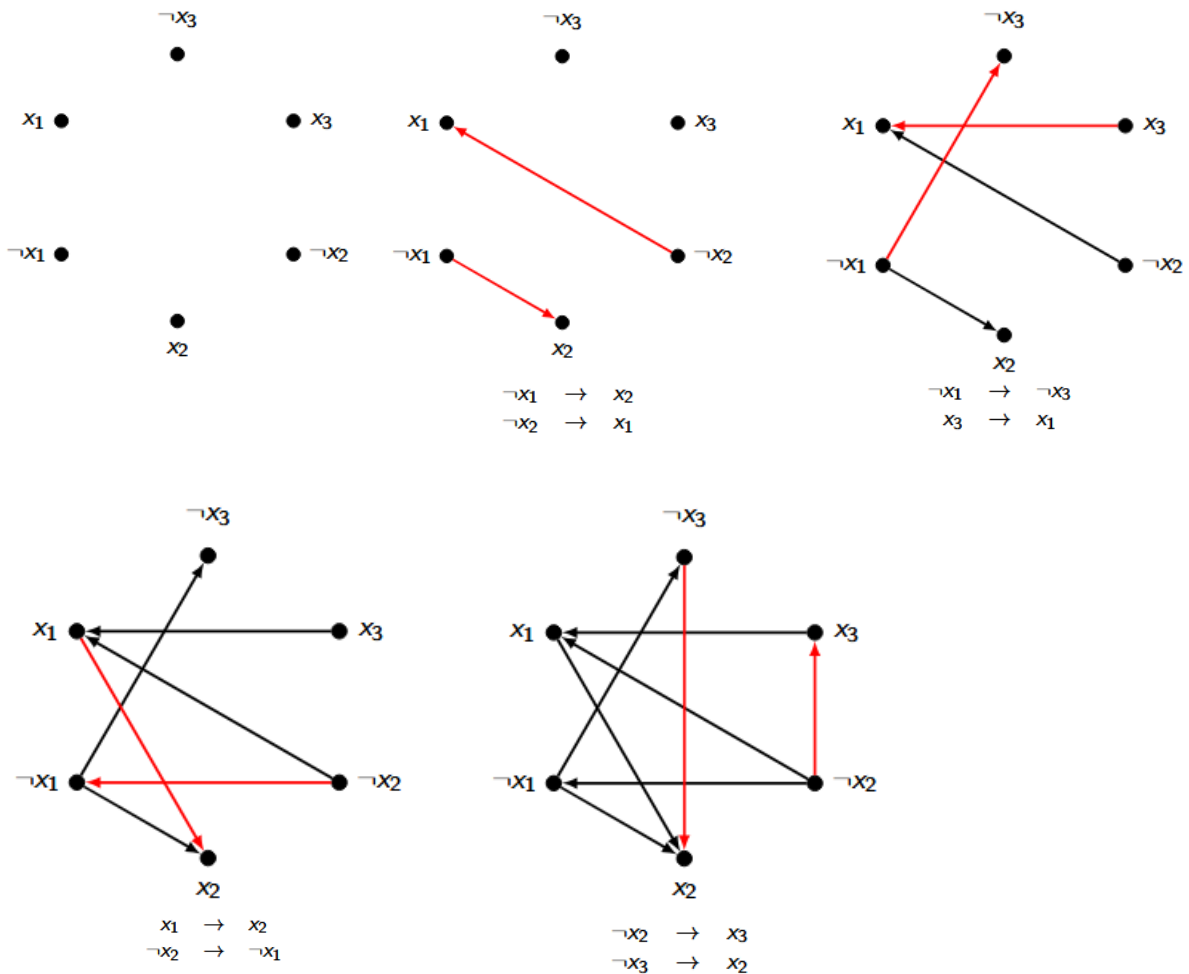
Intuition der Konstruktion

Eine Klausel der Form $(\alpha \vee \beta)$ ist logisch äquivalent zu den Implikationen $\bar{\alpha} \rightarrow \beta$ und $\bar{\beta} \rightarrow \alpha$.

Durch diese Darstellung können wir logische Abhängigkeiten als Pfade im Graphen ausdrücken.

→ Die Erfüllbarkeit der Formel wird durch Erreichbarkeit im Graphen codiert.

Beispiel: $\varphi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$



Zusammenhang zwischen φ und $G\varphi$

Beobachtung (ohne Beweis)

Seien α, β zwei Literale über den Variablen X in φ . Dann gilt die Äquivalenz:

- In jedem Modell I von φ ist $\alpha \rightarrow \beta$ wahr, geschrieben $\varphi \models (\alpha \rightarrow \beta)$
- Im Graphen $G\varphi$ existiert ein Pfad von α nach β .

Problematische Pfade im Graph $G\varphi$

- Frage: Kann φ erfüllbar sein, wenn es in $G\varphi$ einen Pfad von x_i nach $\neg x_i$ gibt?
→ Ja, aber dann muss in jedem Modell $I(x_i) = \text{false}$ gelten.
- Frage: Kann φ erfüllbar sein, wenn es in $G\varphi$ einen Pfad von $\neg x_i$ nach x_i gibt?
→ Ja, aber dann muss in jedem Modell $I(x_i) = \text{true}$ gelten.
- Frage: Was, wenn es beide Pfade $x_i \rightarrow \neg x_i$ und $\neg x_i \rightarrow x_i$ gibt?
→ Dann ist φ unerfüllbar.

Beobachtung (ohne Beweis)

Folgende Behauptungen sind äquivalent:

- (1) Die Formel φ ist erfüllbar.
- (2) Es gibt keine Variable $x_i \in X$, für die der Graph $G\varphi$ sowohl einen Pfad von x_i nach $\neg x_i$ als auch von $\neg x_i$ nach x_i enthält.

- Richtung (1) $\Rightarrow$ (2): folgt direkt aus den vorherigen Überlegungen.
- Richtung (2) $\Rightarrow$ (1): nicht trivial, aber man kann ein Modell I von ϕ konstruieren:
 1. Zuerst werden in I alle Variablen x_i mit false bzw. true belegt, für die es einen Pfad von x_i nach $\neg x_i$ bzw. von $\neg x_i$ nach x_i in G_ϕ gibt.
 2. Dann werden Wahrheitswerte entlang von allen Pfaden propagiert, z. B.:
wenn $I(\alpha) = \text{true}$ gilt und es einen Pfad von α nach β gibt, dann muss auch $I(\beta) = \text{true}$ gelten.
 3. Wenn dann noch Variablen in I unbestimmt sind, kann man für eine dieser Variablen einen beliebigen Wahrheitswert festlegen und diesen dann wieder entlang von Pfaden propagieren.

Damit erhalten wir folgende Turing Reduktion von 2-SAT nach Graph-Erreichbarkeit:

procedure test2SAT:

Input: propositional formula ϕ in 2-CNF;

Output: true if ϕ is satisfiable and false otherwise.

from ϕ construct G_ϕ ;

for all variables x in ϕ **do** {

if G_ϕ contains a path from x to $\neg x$

and a path from $\neg x$ to x **then return** false;}

return true;

Beim Test in der if-Anweisung wird jeweils zwei Mal ein Lösungsverfahren für Graph-Erreichbarkeit als

Unterprozedur aufgerufen, und zwar mit den Instanzen $(G_\phi, x, \neg x)$ und $(G_\phi, \neg x, x)$.

Korrektheit der Reduktion = Korrektheit der Prozedur

test2SAT.

Ein klassisches Entscheidungsproblem der Graphentheorie

Problem k-Färbbarkeit

Sei $k \in \mathbb{N}$ mit $k \geq 2$. Dann ist k-Färbbarkeit folgendermaßen definiert:

- Instanz: ungerichteter Graph $G = (V, E)$

- Frage: Ist der Graph G k-färbbar? d.h.: gibt es eine Funktion $f: V \rightarrow \{0, \dots, k-1\}$, so dass für alle Kanten $[v_i, v_j] \in E$ gilt: $f(v_i) \neq f(v_j)$.

- Man kann k-Färbbarkeit für beliebige Werte $k \geq 2$ betrachten.
- Im Komplexitätsteil der Vorlesung wird gezeigt werden, dass 3-Färbbarkeit (eig. k-Färbbarkeit für jedes $k \geq 3$) ein schweres Problem ist, d.h. es ist kein effizienter Algorithmus bekannt
- Wir betrachten hier 2-Färbbarkeit (ein sehr leichtes Problem).

Theorem

Das 2-Färbbarkeit Problem lässt sich mittels Many-One Reduktion auf das 2-SAT Problem reduzieren.

Problemreduktion

Sei $G = (V, E)$ eine beliebige Instanz von 2-Färbbarkeit, d.h.: $G = (V, E)$ ist ein ungerichteter Graph mit Knotenmenge $V = \{v_1, \dots, v_n\}$ und Kantenmenge E . Daraus definieren wir folgende Instanz ϕ_G von 2-SAT mit den aussagenlogischen Variablen $x_1, \dots, x_n$:

$$\phi_G = \bigwedge_{[v_i, v_j] \in E} ((x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j))$$

- Das 2-SAT-Problem ist effizient lösbar.
- Durch die Reduktion von 2-Färbbarkeit $\rightarrow$ 2-SAT ist auch 2-Färbbarkeit effizient lösbar.
- Reduzierbarkeit ist transitiv:
Kombiniert man die Reduktion von
2-Färbbarkeit $\rightarrow$ 2-SAT und 2-SAT $\rightarrow$ Graph-Erreichbarkeit, erhält man eine Reduktion von
2-Färbbarkeit $\rightarrow$ Graph-Erreichbarkeit.

Korrektheit der Reduktion

Wir müssen folgende Äquivalenz zeigen:

- $G = (V, E)$ ist eine positive Instanz von 2-Färbbarkeit $\Leftrightarrow \varphi_G$ ist eine positive Instanz von 2-SAT.
- Üblicherweise geht man so vor, dass man die beiden Implikationen „ $\Rightarrow$ “ und „ $\Leftarrow$ “ getrennt zeigt.
- Wir zeigen zuerst die Implikation „ $\Rightarrow$ “, d. h.:
 $G = (V, E)$ ist eine positive Instanz von 2-Färbbarkeit $\Rightarrow \varphi_G$ ist eine positive Instanz von 2-SAT.
- Und wir zeigen dann die Implikation „ $\Leftarrow$ “, d. h.:
 $G = (V, E)$ ist eine positive Instanz von 2-Färbbarkeit $\Leftarrow \varphi_G$ ist eine positive Instanz von 2-SAT.

Beweis der „ $\Rightarrow$ “-Richtung

Angenommen $G = (V, E)$ ist eine positive Instanz von 2-Färbbarkeit. Dann gibt es eine Funktion $f: V \rightarrow \{0, 1\}$, die eine gültige 2-Färbung für den Graph $G = (V, E)$ ist, d. h. für jede Kante $[v_i, v_j]$ in G gilt entweder $(f(v_i) = 0 \text{ und } f(v_j) = 1)$ oder $(f(v_i) = 1 \text{ und } f(v_j) = 0)$.

Wir definieren daraus folgende Wahrheitsbelegung I auf den aussagenlogischen Variablen $x_1, \dots, x_n$:

$$I(x_i) = \begin{cases} \text{false, falls } f(v_i) = 0 \\ \text{true, falls } f(v_i) = 1 \end{cases}$$

Wir müssen noch zeigen, dass I die Formel φ_G erfüllt. Die Formel φ_G ist eine Konjunktion von Teilformeln der Form $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$. Es reicht daher zu zeigen, dass I jede Klausel $(x_i \vee x_j)$ und $(\neg x_i \vee \neg x_j)$ erfüllt.

Beweis, dass I jede Klausel $(x_i \vee x_j)$ und $(\neg x_i \vee \neg x_j)$ erfüllt:

1. Beliebige Klausel der Form $(x_i \vee x_j)$:

Laut Problemreduktion enthält φ_G so eine Klausel nur dann, wenn es eine Kante $[v_i, v_j]$ in G gibt. Laut Annahme ist f eine gültige Zweifärbung von G . Also ist mindestens einer der Funktionswerte $f(v_i)$ und $f(v_j)$ gleich 1. Laut Definition von I gilt dann für mindestens eine der Variablen x_i und x_j , dass sie in I den Wahrheitswert true hat. Somit erfüllt I die Klausel $(x_i \vee x_j)$.

2. Beliebige Klausel der Form $(\neg x_i \vee \neg x_j)$:

Laut Problemreduktion enthält φ_G so eine Klausel nur dann, wenn es eine Kante $[v_i, v_j]$ in G gibt. Laut Annahme ist f eine gültige Zweifärbung von G . Also ist mindestens einer der Funktionswerte $f(v_i)$ und $f(v_j)$ gleich 0. Laut Definition von I gilt dann für mindestens eine der Variablen x_i und x_j , dass sie in I den Wahrheitswert false hat. Dann hat aber mindestens eines der Literale $\neg x_i$ und $\neg x_j$ den Wahrheitswert true. Somit erfüllt I die Klausel $(\neg x_i \vee \neg x_j)$.

Beweis der „ $\Leftarrow$ “-Richtung

Angenommen φ_G ist eine positive Instanz von 2-SAT. Dann gibt es eine Wahrheitsbelegung I auf den Variablen $x_1, \dots, x_n$, die die Formel φ_G erfüllt. Die Formel φ_G ist eine Konjunktion von Teilformeln der Form $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$. Wenn I die Formel φ_G erfüllt, dann erfüllt I also auch jede Teilformel $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$ von φ_G . Mit Hilfe von I definieren wir nun folgende Funktion $f: V \rightarrow \{0, 1\}$:

$$f(v_i) = \begin{cases} 0, \text{ falls } I(x_i) = \text{false} \\ 1, \text{ falls } I(x_i) = \text{true} \end{cases}$$

Laut Annahme erfüllt die Wahrheitsbelegung I die Formel φ_G , also auch $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$. Daraus folgt:

- Da $(x_i \vee x_j)$ erfüllt ist, muss mindestens eine der Variablen x_i, x_j in I den Wert true haben.
- Da $(\neg x_i \vee \neg x_j)$ erfüllt ist, muss mindestens eine der Variablen x_i, x_j den Wert false haben.

Somit hat eine Variable den Wert true und die andere false. Nach Definition von f gilt daher $f(v_i) \neq f(v_j)$.

Da dies für alle Kanten $[v_i, v_j]$ gilt, ist f eine gültige 2-Färbung von G .

Typische Vorgangsweise bei Korrektheitsbeweisen

- Eine Many-One-Reduktion R von einem Entscheidungsproblem A auf ein Problem B ist korrekt, wenn gilt: x ist eine positive Instanz von $A \Leftrightarrow R(x)$ ist eine positive Instanz von B .
- Man zeigt die beiden Richtungen „ $\Rightarrow$ “ und „ $\Leftarrow$ “ der Äquivalenz getrennt.
- Der Beweis beginnt meist mit:
 - „Sei x eine positive Instanz von A “ oder
 - „Sei $R(x)$ eine positive Instanz von B “.
- Durch schlüssige Beweisschritte zeigt man, dass auch das andere Problem eine positive Instanz hat.
- Typische Beweisschritte beruhen auf: der Annahme, einer Definition (z. B. einer Lösung) oder der Problemreduktion.
- Zur besseren Lesbarkeit sollte im Beweis immer klar sein, was gerade gezeigt wird und was noch zu zeigen ist.

Unentscheidbarkeitsbeweise mittels Reduktionen

- Das Halteproblem ist unentscheidbar.
 - Es existieren außerdem viele weitere unentscheidbare Probleme (sogar überabzählbar viele).
 - Um die Unentscheidbarkeit eines neuen Problems P zu zeigen, könnte man einen ähnlich komplizierten Beweis wie beim Diagonalargument führen, aber es gibt eine bessere Methode.
 - Idee: Um zu zeigen, dass P unentscheidbar ist, reduzieren wir das Halteproblem (oder ein anderes bekannt unentscheidbares Problem) auf P .
 - Begründung: Angenommen, es gäbe eine Entscheidungsprozedur Π_P für Problem P .
Dann konstruieren wir daraus eine Prozedur Π_h für das Halteproblem:
 - Π_h nimmt eine Instanz $x = (\Pi, I)$ des Halteproblems als Input.
 - Mithilfe der Reduktion R wird daraus eine Instanz $R(x)$ von P berechnet.
 - Π_P wird auf $R(x)$ ausgeführt.
 - Der Output von Π_P ist auch der Output von Π_h .
- Ein Algorithmus für P würde also auch das Halteproblem lösen, was ein Widerspruch ist
- P ist unentscheidbar.

Unentscheidbarkeitsbeweise mittels Reduktionen

Problem KORREKTHEIT

- Instanz: Gegeben ist ein Programm Π sowie zwei Strings I_1 und I_2 .
- Frage: Gefragt ist, ob Π bei Eingabe I_1 den Output I_2 liefert

Theorem

Das Korrektheit Problem ist unentscheidbar.

Beweis mittels Reduktion vom Halteproblem

Sei (Π, I) eine beliebige Instanz des Halteproblems, d. h.: Π ist ein SIMPLE-Programm und I ist ein möglicher Input-String für Π . Daraus definieren wir eine Instanz (Π', I_1, I_2) des Korrektheit-Problems, indem wir $I_1 = I_2 = I$ setzen und Π' wie folgt definieren:

```
String  $\Pi'$  (String S) {  
    call  $\Pi(S)$ ;  
    return S;  
}
```

Wir müssen die Korrektheit der Reduktion zeigen, also:

(Π, I) ist eine positive Instanz des Halteproblems $\Leftrightarrow (\Pi', I_1, I_2)$ ist eine positive Instanz des Korrektheit-Problems.

Beweis „ $\Rightarrow$ “

Angenommen, (Π, I) ist eine positive Instanz des Halteproblems, d. h. Π hält bei Input I . Dann hält auch Π' auf I_1 und liefert $I_2 = I$ als Output.

$\rightarrow (\Pi', I_1, I_2)$ ist eine positive Instanz des Korrektheit-Problems.

Beweis „ $\Leftarrow$ “

Angenommen, (Π', I_1, I_2) ist eine positive Instanz des Korrektheit-Problems. Dann hält Π' auf $I_1 = I_2 = I$ und ruft dabei $\Pi(I)$ auf. Da Π' hält, hält auch Π .

$\rightarrow (\Pi, I)$ ist eine positive Instanz des Halteproblems.

Problem SELBER OUTPUT

Instanz: Programme Π_1, Π_2 und Input String I .

Frage: Haben Π_1 und Π_2 auf Input I das gleiche Verhalten?

Theorem

Das SELBER-Output Problem ist unentscheidbar.

Beweis mittels Reduktion vom Halteproblem

Sei (Π, I) eine beliebige Instanz des Halteproblems, d. h. Π ist ein SIMPLE-Programm und I ist ein möglicher Input-String für Π . Daraus definieren wir eine Instanz (Π_1, Π_2, I') des Selber-Output-Problems mit $I' = I$ und folgenden Programmen Π_1 und Π_2 :

```
String  $\Pi_1$  (String S) {  
    return S;  
}
```

```
String  $\Pi_2$  (String S) {  
    call  $\Pi(S)$ ;  
    return S;  
}
```

Intuition der Reduktion:

- Programm Π_1 liefert bei Input I sofort den Output $I \rightarrow$ es hält immer.
- Programm Π_2 ruft Π mit Input I auf und gibt I als Output aus – falls Π bei Input I hält.

Wir müssen die Korrektheit der Reduktion zeigen, d. h.:

(Π, I) ist eine positive Instanz des Halteproblems $\Leftrightarrow (\Pi_1, \Pi_2, I)$ ist eine positive Instanz des Selber-Output-Problems. Beide Richtungen der Äquivalenz:

Beweis „ $\Rightarrow$ “

Wenn Π bei Input I hält, terminiert auch Π_2 bei I und liefert I als Output. Da Π_1 immer bei I hält und I ausgibt, gilt: (Π_1, Π_2, I) ist eine positive Instanz des Selber-Output-Problems.

Beweis „ $\Leftarrow$ “

Wenn (Π_1, Π_2, I) eine positive Instanz des Selber-Output-Problems ist, dann terminiert Π_2 bei $I \rightarrow$ somit hält Π bei Input I . Also ist (Π, I) eine positive Instanz des Halteproblems.

Beweise der Nicht-Semi-Entscheidbarkeit

Idee:

- Wir wissen, dass das co-Halteproblem nicht semi-entscheidbar ist.
- Daher kann man die Nicht-Semi-Entscheidbarkeit eines neuen Problems durch eine Reduktion vom co-Halteproblem beweisen.

Beobachtung:

- Angenommen, wir können für zwei Probleme zeigen, dass $P_1 \leq P_2$ gilt. Dann gilt offensichtlich auch $\text{co-}P_1 \leq \text{co-}P_2$.

Begründung:

- Korrektheit einer Reduktion R bedeutet:
Für jede Instanz x gilt x ist positive Instanz von $P_1 \Leftrightarrow R(x)$ ist positive Instanz von P_2 .
- Dann gilt auch:
 x ist negative Instanz von $P_1 \Leftrightarrow R(x)$ ist negative Instanz von P_2 .
bzw. positive Instanz von $\text{co-}P_1 \Leftrightarrow$ positive Instanz von $\text{co-}P_2$
 $\rightarrow$ Daher gilt: $P_1 \leq P_2 \Leftrightarrow \text{co-}P_1 \leq \text{co-}P_2$.

Schlussfolgerung: Die Probleme co-Korrektheit und co-Selber-Output sind nicht semi-entscheidbar.

Problem SELBER OUTPUT

Instanz: Programme Π_1, Π_2 und Input String I .

Frage: Haben Π_1 und Π_2 auf Input I das gleiche Verhalten?

Theorem

Das SELBER-Output Problem ist nicht semi-entscheidbar.

Beweis mittels Reduktion vom co-Halteproblem

Sei (Π, I) eine beliebige Instanz des co-Halteproblems, d. h. Π ist ein SIMPLE-Programm und I ist ein möglicher Input-String für Π . Daraus definieren wir eine Instanz (Π_1, Π_2, I') des Selber-Output-Problems mit $I' = I$ und folgenden Programmen Π_1 und Π_2 :

String Π_1 (String S) {	String Π_2 (String S) {
While (true) do {};	call $\Pi(S)$; return S ; }

Intuition der Reduktion:

- Π_1 ignoriert den Input I und läuft sofort in eine Endlosschleife, d. h. Π_1 terminiert nicht.
- Π_2 ruft Π mit Input I auf und gibt anschließend I als Output aus, also terminiert Π_2 genau dann, wenn Π auf I terminiert.

Wir müssen die Korrektheit der Reduktion zeigen, d. h.:

(Π, I) ist eine positive Instanz des co-Halteproblems $\Leftrightarrow (\Pi_1, \Pi_2, I)$ ist eine positive Instanz des Selber-Output-Problems. Beide Richtungen der Äquivalenz:

Beweis „ $\Rightarrow$ “

Wenn Π bei Input I nicht hält, dann terminiert auch Π_2 nicht. Somit haben Π_1 und Π_2 bei I das gleiche Verhalten (beide laufen endlos).

$\rightarrow (\Pi_1, \Pi_2, I)$ ist eine positive Instanz des Selber-Output-Problems.

Beweis „ $\Leftarrow$ “

Wenn (Π_1, Π_2, I) eine positive Instanz des Selber-Output-Problems ist, dann haben Π_1 und Π_2 identisches Verhalten. Da Π_1 nie terminiert, gilt: Π_2 (und damit Π) terminiert bei I nicht.

$\rightarrow (\Pi, I)$ ist eine positive Instanz des co-Halteproblems.